

GUIA DE DEPLOY

Agente de Atendimento IA

Passo a passo completo para configurar, testar e colocar o agente em produção — Docker ou processo direto, Telegram e/ou WhatsApp — com checklist de lançamento e rotina de monitoramento pós-deploy.

Setup local

Docker

Telegram + WhatsApp

Checklist de produção

Repositório: github.com/andre-rizzato/ai-customer-service-agent

Versão do documento: 1.0 — Setembro/2026

Sumário

1. Pré-requisitos
2. Variáveis de ambiente (referência completa)
3. Setup local passo a passo
4. Deploy com Docker
5. Deploy sem Docker (processo direto / PM2)
6. Configurar o canal Telegram
7. Configurar o canal WhatsApp
8. Checklist completo de lançamento
9. Monitoramento pós-deploy
10. Troubleshooting comum
11. Segurança e segredos

1. Pré-requisitos

Item	Necessário para
Node.js 18.17+ (recomendado 22)	Rodar o servidor sem Docker; sempre necessário para <code>npm run ingest</code>
Docker + Docker Compose (opcional)	Deploy containerizado (Seção 4)
Chave de API Anthropic ou OpenAI	Gerar as respostas do agente (<code>LLM_PROVIDER</code>)
Chave de API Voyage AI (ou OpenAI, ou nenhuma p/ modo local)	Gerar embeddings do catálogo e das perguntas (<code>EMBEDDING_PROVIDER</code>)
Token de bot do Telegram (@BotFather)	Canal Telegram (opcional)
App configurado no Meta for Developers	Canal WhatsApp (opcional)
URL pública (domínio próprio ou túnel como ngrok em dev)	Receber webhooks de Telegram/WhatsApp

2. Variáveis de ambiente (referência completa)

Todas validadas por `src/config.ts` na inicialização — o processo recusa subir se algo obrigatório faltar. Copie `.env.example` para `.env` e preencha.

Variável	Obrigatória quando	Descrição
<code>PORT</code>	Nunca (default 3000)	Porta HTTP do servidor
<code>AGENT_CONFIG_PATH</code>	Nunca (default <code>./config/agent.config.json</code>)	Caminho do arquivo de configuração do negócio
<code>LLM_PROVIDER</code>	Nunca (default <code>anthropic</code>)	<code>anthropic</code> ou <code>openai</code>
<code>ANTHROPIC_API_KEY</code>	<code>LLM_PROVIDER=anthropic</code>	Chave da API da Anthropic
<code>ANTHROPIC_MODEL</code>	Nunca (default <code>claude-haiku-4-5-20251001</code>)	Modelo Claude a usar
<code>OPENAI_API_KEY</code>	<code>LLM_PROVIDER=openai</code> ou <code>EMBEDDING_PROVIDER=openai</code>	Chave da API da OpenAI
<code>OPENAI_MODEL</code>	Nunca (default <code>gpt-4o-mini</code>)	Modelo de chat da OpenAI
<code>EMBEDDING_PROVIDER</code>	Nunca (default <code>voyage</code>)	<code>voyage</code> , <code>openai</code> ou <code>local</code>
<code>VOYAGE_API_KEY</code>	<code>EMBEDDING_PROVIDER=voyage</code>	Chave da API da Voyage AI
<code>VOYAGE_MODEL</code>	Nunca (default <code>voyage-3.5-lite</code>)	Modelo de embedding da Voyage
<code>OPENAI_EMBEDDING_MODEL</code>	Nunca (default <code>text-embedding-3-small</code>)	Modelo de embedding da OpenAI
<code>TELEGRAM_BOT_TOKEN</code>	Para habilitar o canal Telegram	Token do @BotFather
<code>TELEGRAM_WEBHOOK_SECRET</code>	Recomendado em produção	Autentica que o webhook veio do Telegram
<code>WHATSAPP_ACCESS_TOKEN</code>	Para habilitar o canal WhatsApp	Token de acesso da Graph API
<code>WHATSAPP_PHONE_NUMBER_ID</code>	Para habilitar o canal WhatsApp	Id do número no Meta for Developers
<code>WHATSAPP_VERIFY_TOKEN</code>	Para habilitar o canal WhatsApp	Token definido por você para o handshake de verificação
<code>HANDOFF_WEBHOOK_URL</code>	<code>agent.config.json</code> → <code>handoffNotifier: "webhook"</code>	URL (ex.: Incoming Webhook do Slack) para alertas de handoff
<code>CONVERSATIONS_DIR</code>	Nunca (default <code>./data/conversations</code>)	Pasta com histórico por conversa
<code>AUDIT_LOG_PATH</code>	Nunca (default <code>./data/audit-log.jsonl</code>)	Log de auditoria (JSON Lines)

Persistência dos dados

`CONVERSATIONS_DIR` e `AUDIT_LOG_PATH` (e o índice vetorial em `config.vectorStorePath`) vivem em disco local. Em qualquer deploy containerizado ou com múltiplas réplicas, isso PRECISA ser um volume persistente — sem isso, um redeploy apaga todo o histórico e o índice do catálogo.

3. Setup local passo a passo

- 1 Clonar o repositório e instalar dependências: `npm install`
- 2 Copiar os templates: `cp .env.example .env` e `cp config/agent.config.example.json config/agent.config.json`
- 3 Preencher `.env` com as chaves de LLM/embeddings (Seção 2) e `config/agent.config.json` com nome do negócio, tom de voz e gatilhos de handoff (Fase 0: defina isso com o negócio antes de codar)
- 4 Editar/substituir `knowledge/catalog.example.json` pelo catálogo real (uma linha por produto/serviço/política)
- 5 Indexar a base: `npm run ingest` — repita sempre que o catálogo mudar
- 6 Testar sem canal nenhum: `npm run simulate` — rode a matriz de testes da Fase 5 (dentro do escopo, fora do escopo, "você é IA?", pedido de humano, reclamação, mensagem ambígua)
- 7 Rodar a suíte automatizada: `npm test`
- 8 Só depois de validar os passos 6 e 7, seguir para a Seção 4 ou 5 (deploy)

4. Deploy com Docker

O `Dockerfile` na raiz do projeto faz build multi-stage: compila TypeScript em um estágio com as devDependencies, e a imagem final roda só o JavaScript compilado com `node` puro (sem `tsx`, sem TypeScript instalado) — imagem menor e sem ferramentas de build na superfície de produção.

```
# build da imagem
docker build -t ai-customer-service-agent .

# rodar com variáveis de ambiente e volume de dados persistente
docker run -d \
  --name agente-atendimento \
  -p 3000:3000 \
  --env-file .env \
  -v $(pwd)/data:/app/data \
  ai-customer-service-agent
```

Catálogo e configuração customizados na imagem

O Dockerfile copia `config/agent.config.example.json` e `knowledge/catalog.example.json` como fallback funcional. Para produção, monte seus arquivos reais por volume (`-v $(pwd)/config/agent.config.json:/app/config/agent.config.json` e o equivalente para o catálogo) ou construa uma imagem própria por cliente copiando os arquivos reais no lugar dos exemplos.

Depois de subir o container, rode o ingest uma vez (dentro do container ou apontando `VECTOR_STORE` /dados para o mesmo volume montado):

```
docker exec -it agente-atendimento node dist/src/knowledge/ingest.js
```

docker-compose (opcional)

```
services:
  agente:
    build: .
    ports: ["3000:3000"]
    env_file: .env
    volumes:
      - ./data:/app/data
      - ./config/agent.config.json:/app/config/agent.config.json
      - ./knowledge/catalog.json:/app/knowledge/catalog.json
    restart: unless-stopped
```

5. Deploy sem Docker (processo direto / PM2)

Alternativa em uma VPS comum, sem containers:

- 1 Na VPS: `git clone`, `npm ci`, configurar `.env` e `config/agent.config.json`
- 2 Compilar: `npm run build`
- 3 Rodar o processo com um gerenciador que reinicia sozinho em caso de crash — `pm2 start dist/src/server.js --name agente-atendimento` (ou um serviço `systemd` equivalente)
- 4 Colocar um proxy reverso (Nginx/Caddy) na frente, com TLS, apontando para a porta do processo (`PORT` no `.env`)
- 5 Rodar `npm run ingest` uma vez após o deploy (e a cada atualização do catálogo)

6. Configurar o canal Telegram

- 1 Criar um bot com o **@BotFather** no Telegram e copiar o token gerado
- 2 Definir `TELEGRAM_BOT_TOKEN` (e um `TELEGRAM_WEBHOOK_SECRET` à sua escolha) no `.env`
- 3 Com o servidor já publicado em uma URL HTTPS pública, registrar o webhook:

```
curl "https://api.telegram.org/bot<TOKEN>/setWebhook?url=<SUA-URL-PUBLICA>/webhook/telegram&secret_token=<TELEGRAM_WEBHOOK_SECRET>"
```

- 4 Confirmar: mande uma mensagem para o bot no Telegram e verifique se a resposta chega

Testando localmente antes de ter domínio público

Use um túnel (ex.: `ngrok http 3000`) para gerar uma URL HTTPS temporária apontando para sua máquina local, e use essa URL no `setWebhook` acima.

7. Configurar o canal WhatsApp (Meta Cloud API)

- 1 Criar um app no **Meta for Developers** e adicionar o produto WhatsApp
- 2 Copiar `WHATSAPP_ACCESS_TOKEN` e `WHATSAPP_PHONE_NUMBER_ID` do painel do app
- 3 Definir `WHATSAPP_VERIFY_TOKEN` — uma string à sua escolha, usada só no handshake de verificação
- 4 No painel do Meta, configurar o webhook apontando para `<SUA-URL-PUBLICA>/webhook/whatsapp`, usando o mesmo `WHATSAPP_VERIFY_TOKEN` — a Meta faz uma chamada GET de verificação automaticamente
- 5 Assinar o webhook para o campo `messages`
- 6 Confirmar: mande uma mensagem de teste para o número WhatsApp Business e verifique a resposta

Número verificado para clientes reais

Para atendimento comercial de verdade, use um número WhatsApp Business verificado, conectado ao webhook de PRODUÇÃO — separado do número/ambiente usado nos testes acima (Fase 6 do runbook original).

8. Checklist completo de lançamento

Antes de codar (Fase 0)

- ☐ Escopo fechado: 10–20 perguntas frequentes reais do negócio listadas
- ☐ Histórico de 15–20 conversas reais revisado (tom de voz, jeito de falar)
- ☐ Critério de sucesso mensurável definido (taxa de resolução, tempo de resposta)

Configuração

- ☐ `.env` preenchido com chaves de LLM e embeddings válidas
- ☐ `config/agent.config.json` preenchido (nome do negócio, tom, gatilhos de handoff, rate limit)
- ☐ `knowledge/catalog.json` com o catálogo real do negócio (não o example)
- ☐ `npm run ingest` executado com sucesso após o catálogo real

Testes (Fase 5 — não pular)

- ☐ `npm test` passando (handoff, prompt, rate limit, vector store)
- ☐ `npm run simulate` : pergunta dentro do escopo respondida corretamente com base no catálogo
- ☐ `npm run simulate` : pergunta fora do escopo → bot admite que não sabe, nunca inventa
- ☐ `npm run simulate` : "você é uma IA?" → resposta de transparência total
- ☐ `npm run simulate` : pedido explícito de atendente → handoff dispara
- ☐ `npm run simulate` : mensagem de reclamação/frustração → handoff dispara
- ☐ Matriz de testes rodada por 2+ pessoas diferentes (evita viés de quem escreveu o prompt)

Infraestrutura

- ☐ Volume/persistência configurado `data/` (histórico + auditoria + índice vetorial sobrevivem a redeploy)
- ☐ HTTPS válido na URL pública do servidor (obrigatório para Telegram/WhatsApp aceitarem o webhook)
- ☐ Rate limit ajustado em `agent.config.json` conforme volume esperado
- ☐ Alerta de handoff configurado — `handoffNotifier: "webhook"` + `HANDOFF_WEBHOOK_URL` apontando para Slack/canal do time
- ☐ Health check (`/health`) integrado ao monitoramento de infraestrutura

Canais

- ☐ Webhook do canal configurado e testado isoladamente (mensagem simples, sem LLM, se possível)
- ☐ Número/bot de PRODUÇÃO separado do ambiente de teste
- ☐ `TELEGRAM_WEBHOOK_SECRET` configurado (Telegram) — evita webhook falso

Lançamento

- ☐ Soft-launch: só uma fração dos contatos (ex.: novos leads) passa pelo bot na primeira semana
- ☐ Atendimento humano continua paralelo durante o soft-launch
- ☐ Rotina semanal de auditoria de amostra agendada (Fase 7)

Não pule o soft-launch

Ir direto para 100% do volume é o erro que transforma um problema de prompt em problema de reputação — um cliente que recebe resposta errada em massa não volta.

9. Monitoramento pós-deploy

Rotina	Frequência	Onde/como
Auditoria de amostra (ler conversas, marcar respostas duvidosas)	Semanal	<code>data/audit-log.jsonl</code> — cada linha tem <code>contextUsed</code> mostrando o(s) item(ns) da base usados naquela resposta
Atualizar o catálogo sempre que produto/preço/política mudar	Sob demanda	Editar <code>knowledge/catalog.json</code> + rodar <code>npm run ingest</code> de novo
Revisar taxa de handoff (alto demais / baixo demais)	Semanal/quinzenal	Ajustar <code>handoffKeywords</code> / <code>frustrationKeywords</code> / <code>minRelevanceScore</code> em <code>agent.config.json</code>
Checar health check e uso de CPU/memória do processo	Contínuo	Ferramenta de monitoramento de infraestrutura apontando para <code>/health</code>

10. Troubleshooting comum

Sintoma	Causa provável	O que checar
Processo não sobe, erro de validação na inicialização	Variável de ambiente ou campo de <code>agent.config.json</code> faltando/errado	Ler a mensagem de erro (o zod aponta exatamente o campo) e comparar com <code>.env.example</code>
Bot sempre responde "não tenho essa informação"	Índice vetorial vazio ou desatualizado	Rodar <code>npm run ingest</code> ; checar se <code>knowledgeBasePath</code> / <code>vectorStorePath</code> apontam para os arquivos certos
Webhook do Telegram/WhatsApp não recebe nada	URL pública não é HTTPS válido, ou webhook não foi registrado	Repetir o passo de <code>setWebhook</code> (Telegram) ou a configuração no Meta for Developers (WhatsApp)
WhatsApp: verificação do webhook falha (403)	<code>WHATSAPP_VERIFY_TOKEN</code> diferente do configurado no painel Meta	Conferir se o valor no <code>.env</code> é idêntico ao digitado no painel
Handoff nunca dispara	Palavras-chave configuradas não cobrem o vocabulário real dos clientes	Revisar <code>handoffKeywords</code> / <code>frustrationKeywords</code> com base em conversas reais
Handoff dispara demais	Palavra-chave genérica demais capturando mensagens normais	Tornar as frases mais específicas em vez de palavras soltas
Custo de LLM/embeddings mais alto que o esperado	Rate limit frouxo, ou muitas mensagens repetidas de um mesmo usuário	Apertar <code>rateLimit</code> em <code>agent.config.json</code>
Histórico "sumiu" após redeploy	<code>data/</code> não estava em um volume persistente	Configurar volume (Docker) ou pasta persistente fora do diretório de deploy (processo direto)

11. Segurança e segredos

- `.env` nunca deve ser commitado — já está no `.gitignore` do projeto.
- Use `TELEGRAM_WEBHOOK_SECRET` em produção — sem ele, qualquer pessoa que descubra a URL do webhook pode enviar mensagens falsas para o agente processar.
- A verificação do webhook do WhatsApp (`WHATSAPP_VERIFY_TOKEN`) só protege o handshake inicial — a Meta não assina cada mensagem individualmente por padrão neste fluxo simples; trate a URL do webhook como não-pública mesmo assim.
- Rotacione chaves de API (Anthropic/OpenAI/Voyage) periodicamente e imediatamente se algum `.env` vazar acidentalmente.
- O log de auditoria (`data/audit-log.jsonl`) contém texto de conversas reais de clientes — trate como dado sensível (acesso restrito, backups seguros, conformidade com LGPD se aplicável).